

U1T01: Real-Time Analytics with PostgreSQL CDC & ClickHouse

Modern data architectures require both transactional and analytical workloads. As not a single database engine can cover both scenarios, often companies rely on having one type.

An interesting question emerges; how do we keep both database systems on sync? Most modern cloud warehouses already provide solutions to this, yet it is fundamental to give a taste on how this works under the hood.

Change Data Capture (CDC) is a widely used approach for this. As ETL approaches require constant scheduling to push data between systems, CDC should be designed to push data on (hopefully most of the cases) real-time.

In this assignment, your main task is to implement CDC between PostgreSQL (OLTP) and Clickhouse (OLAP) and test it with a continuous ingestion of data between systems. You will also run some queries in both database systems to compare query plans and benchmark performance, so you get an empirical understanding about which system is best for what.

Part 1: Relational Modeling & Data Generator.

Deploy a PostgreSQL database.

Design a normalized schema (3FN) for fast-paced domains like e-commerce, ridesharing/door-dashing, sensor telemetry, or else. The schema must include at least three tables with primary and foreign keys.

Write a script that continuously executes INSERT, UPDATE, and DELETE operations. Also, you should try to get at least a couple of hundred thousands of operations in your database.

Part 2: CDC Pipeline Set Up.

Configure PostgreSQL and use PeerDB to set up a CDC pipeline.

PostgreSQL configuration to enable logical replication. This may include several steps inside of PostgreSQL, like adding new setups, roles, and services.

PeerDB is recommended, but you are allowed to use any other open-source tool available. For PeerDB you may need to spin up another container, add some configs. I truly recommend reading some PeerDB documentation.

Part 3: Analytical Layer Set Up.

Spin up a ClickHouse instance via Docker.

Recreate the target tables in ClickHouse.

Since ClickHouse is an OLAP database, it may not handle updates and deletes exactly like PostgreSQL. Tip: Take a look at Clickhouse command FINAL.

Verify that an update made by the Python generator in PostgreSQL correctly updates the final state of that record in ClickHouse.

Part 4: Benchmarking.

Write 3 to 5 complex analytical queries. These should involve aggregations, grouping, and filtering across large datasets.

Run the exact same analytical queries against both PostgreSQL and ClickHouse while the data generator is actively pumping new data

Use EXPLAIN ANALYZE (PostgreSQL) and EXPLAIN (ClickHouse) to capture execution plans.

In the PDF report, compare execution times, CPU/memory behavior (if observable), and explain why the query plans look different (if applicable).

Part 5: Real-Time dashboard (optional).

Add a lightweight visualization (streamlit, grafana, else) and connect it directly to ClickHouse. Demonstrate that as the Python generator pushes a transaction to Postgres, it reflects on the ClickHouse-backed dashboard almost instantly.

Deliverables

- Code (zip, GitHub repo, or else). It all should run with a single docker compose command.
- Report: ER model, steps/considerations to implement CDC, benchmarking results.